

대학생 갓생 살기 컨셉에 맞춰서, 할 일 개수와 집중 시간을 조합해 성취 점수를 계산하는 재미있는 로직을 짜줘. 1학년 수준의 리스트와 조건문을 활용하고 싶어



대학생 '갓생(God+生)' 살기 프로젝트의 핵심은 **효율성**과 **몰입**이죠! 1학년 수준에서도 충분히 구현할 수 있도록 파이썬의 ****리스트(List)****와 ****조건문(if-elif-else)****을 활용한 성취도 계산 로직을 짜보았습니다.

단순히 할 일을 많이 하는 게 아니라, 얼마나 집중했느냐에 따라 보너스 점수를 주는 것이 포인트입니다.

갓생 성취도 계산기 로직

1. 데이터 구조 설정

먼저 오늘 끝낸 할 일들을 리스트에 담고, 총 집중 시간을 변수에 저장합니다.

python



```
# 오늘 완료한 할 일 리스트
todo_list = ["전공 수업 복습", "파이썬 공부", "운동하기", "독서"]

# 오늘 총 집중 시간 (단위: 분)
focus_time = 150

# 기본 점수 설정
task_count = len(todo_list)
score = 0
```

2. 점수 계산 조건문

할 일 개수에 따른 기본 점수와 집중 시간에 따른 배율(Multiplier)을 적용합니다.

python



```
# [단계 1] 할 일 개수 점수 (개당 10점)
score = task_count * 10

# [단계 2] 집중 시간에 따른 성취 등급과 보너스 배율
if focus_time >= 180:
    rank = "갓생 전문가"
    multiplier = 1.5
elif focus_time >= 120:
    rank = "성실한 갓생러"
    multiplier = 1.2
elif focus_time >= 60:
    rank = "입문 갓생러"
    multiplier = 1.0
else:
    rank = "갓생 꿈나무"
    multiplier = 0.8
```



Gemini에게 물어보세요



빠른 모델



```
# [단계 3] 최종 점수 계산
final_score = score * multiplier
```

3. 결과 출력

사용자에게 재미있는 피드백을 남겨줍니다.

python



```
print(f"--- 오늘의 갯생 보고서 ---")
print(f"완료한 할 일: {task_count}개")
print(f"총 집중 시간: {focus_time}분")
print(f"당신의 등급: {rank}")
print(f"최종 성취 점수: {final_score}점")

# 특별 응원 메시지
if final_score >= 50:
    print("오늘 정말 고생했어요! 완벽한 갯생입니다. ✨")
else:
    print("내일은 조금 더 몰입해볼까요? 화이팅! 🔥")
```

로직의 재미 포인트 (Level Up!)

- **가중치(Multiplier) 시스템:** 집중 시간이 너무 적으면(60분 미만) 오히려 배율이 0.8로 깎이게 설정했습니다. "양보다는 질"을 강조하는 로직이에요.
- **리스트 활용:** len(todo_list)를 사용하면 리스트에 항목을 추가하거나 삭제할 때마다 점수가 자동으로 변하기 때문에 관리하기 편합니다.
- **확장성:** 나중에 리스트 안에 ["전공", 30]처럼 **[할 일, 소요시간]** 형태의 2차원 리스트를 넣으면 훨씬 복잡하고 정교한 계산기도 만들 수 있습니다.

이 코드로 오늘 하루를 점수화해보면 공부하는 재미가 훨씬 쏠쏠할 거예요! 더 추가하고 싶은 기능이 있나요?

👍 🗨️ 📌 📄 :

리스트, 조건문, 반복문을 배운 단계에서 이 프로그램을 어떻게 확장하면 좋을까 4단계 로드맵을 제안해줘.



리스트, 조건문, 반복문을 모두 마스터하셨군요! 이제 프로그램은 단순한 '계산기'에서 데이터를 관리하는

Gemini에게 물어보세요

+ ☰

빠른 모델



리스트, 조건문, 반복문을 배운 단계에서 이 프로그램을 어떻게 확장하면 좋을까 4단계 로드맵을 제안해줘.



리스트, 조건문, 반복문을 모두 마스터하셨군요! 이제 프로그램은 단순한 '계산기'에서 데이터를 관리하는 '시스템'으로 진화할 수 있습니다. 갓생 살기 컨셉을 유지하면서 실력을 키울 수 있는 **4단계 확장 로드맵**을 제안해 드릴게요.

🚀 갓생 프로그램 진화 로드맵

1단계: 반복문으로 할 일 '직접' 입력받기 (while, append)

지금은 리스트를 미리 만들어두었지만, 이제는 프로그램 실행 중에 할 일을 하나씩 추가할 수 있게 만듭니다.

- **핵심 기능:** while 문을 사용해 "그만"이라고 입력할 때까지 할 일을 input()으로 받고, append()를 이용해 리스트에 추가합니다.
- **학습 포인트:** 무한 루프 제어와 리스트 동적 추가.

2단계: 개별 할 일마다 중요도 부여하기 (중첩 리스트)

모든 할 일이 10점인 것은 아쉽죠? 전공 공부는 20점, 방 정리는 5점처럼 중요도를 다르게 설정해 보세요.

- **핵심 기능:** [["전공 공부", 20], ["물 마시기", 5]]와 같이 리스트 안에 리스트를 넣는 구조를 활용합니다.
- **학습 포인트:** 2차원 리스트 인덱싱과 반복문(for todo in total_list)을 통한 데이터 추출.

3단계: 누적 데이터로 '주간 통계' 내기 (리스트 누적)

오늘의 점수만 보는 게 아니라, 일주일 동안의 점수를 모아서 평균을 내봅니다.

- **핵심 기능:** 7일간의 점수를 저장하는 weekly_scores 리스트를 만들고, sum()과 len()을 활용해 평균 점수를 계산합니다.
- **학습 포인트:** 리스트 내장 함수 활용 및 데이터 요약 기술.

4단계: 갓생 랭킹 및 칭호 부여 시스템 (심화 조건문)

단순 등급을 넘어, 특정 조건을 만족할 때 특별한 '업적'을 잠금 해제하는 기능을 넣습니다.

- **핵심 기능:** * 3일 연속 80점 이상이면 "갓생 폭주 기관차" 칭호 부여.
 - 중요도가 높은 일(20점 이상)을 3개 이상 "완료"하면 "프로 효율러" 보너스.
- **학습 포인트:** 복합 조건문(and, or)과 플래그(flag) 변수 활용법.



Gemini에게 물어보세요



- 학습 포인트: 복합 조건문(and, or)과 플래그(flag) 변수 활용법.

💡 로드맵 요약 가이드

단계	추가되는 문법	프로그램의 변화
Step 1	while, append	내가 직접 할 일을 입력하는 인터랙티브한 앱이 됨
Step 2	중첩 리스트	할 일마다 점수가 다른 정교한 계산기
Step 3	for, 리스트 함수	하루가 아닌 '한 주'를 관리하는 플래너
Step 4	복합 조건문	게임처럼 성취감을 주는 갓생 게임 시스템

이 중에서 가장 먼저 구현해보고 싶은 단계는 무엇인가요? 1단계 코드를 짜는 데 도움이 필요하다면 바로 말주세요!

👍 🗨️ 📌 📄 ⋮

while은 아직 안 배웠어



아하, while은 아직 배우기 전이시군요! 그렇다면 이미 배운 **for 반복문**과 **리스트**, **조건문**만 사용해서 프로그램을 훨씬 멋지게 업그레이드할 수 있습니다.

while 없이도 사용자에게 "오늘 할 일이 총 몇 개인가요?"라고 물어본 뒤, 그 개수만큼 반복해서 입력받는 방식이면 충분히 구현 가능합니다.

🔧 1단계 업그레이드: for 문으로 입력받기

사용자가 미리 정해진 개수만큼 할 일을 직접 입력하고, 그 내용이 리스트에 차곡차곡 쌓이는 로직입니다.

python



```
# 1. 오늘 할 일의 총 개수를 입력받기
total_count = int(input("오늘 완료한 할 일 총 몇 개인가요? "))

todo_list = []
```



Gemini에게 물어보세요



빠른 모델



```
total_count = int(input("오늘 완료한 할 일은 총 몇 개인가요? "))

todo_list = []

# 2. 입력받은 개수만큼 for 반복문 실행
for i in range(total_count):
    task = input(f"{i+1}번째 할 일을 입력하세요: ")
    todo_list.append(task) # 리스트에 추가

# 3. 집중 시간 입력받기
focus_time = int(input("총 집중 시간(분)을 입력하세요: "))

# --- 여기서부터는 아까 짰던 점수 계산 로직 ---
task_count = len(todo_list)
score = task_count * 10

if focus_time >= 120:
    multiplier = 1.2
else:
    multiplier = 1.0

final_score = score * multiplier

print(f"\n✅ 리스트 확인: {todo_list}")
print(f"🌟 최종 점수: {final_score}점!")
```

🚀 while 없이 가는 4단계 로드맵 (수정 버전)

for 반복문의 특성(정해진 횟수만큼 반복)을 활용한 로드맵으로 다시 제안해 드릴게요.

1단계: 동적 리스트 생성 (for + append)

- **내용:** 위 코드처럼 range()와 input()을 조합해 프로그램 실행 중에 리스트를 완성하기.
- **포인트:** 리스트가 비어있다가([]) 하나씩 채워지는 원리를 이해합니다.

2단계: 할 일 목록 열람하기 (for element in list)

- **내용:** 점수 계산이 끝나고 마지막에 for task in todo_list:를 사용하여 오늘 내가 한 일들을 예쁘게 번호를 매겨 출력해 봅니다.
- **포인트:** 리스트에 담긴 데이터를 하나씩 꺼내 쓰는 법을 익힙니다.

3단계: 특정 조건 항목 걸러내기 (for + if)

- **내용:** 입력받은 할 일 중에 '운동'이나 '독서' 같은 특정 단어가 포함되어 있으면 가산점(+5점)을 주는 로직을 추가합니다.
- **포인트:** 반복문 안에서 조건문을 사용해 데이터를 검사하는 법을 배웁니다.

4단계: 점수 히스토리 관리 (중첩 리스트)

- **내용:** scores = [80, 95, 70] 처럼 며칠간의 점수를 리스트에 담고, for 문으로 이 점수들을 다 더해서 평균을 냅니다.
- **포인트:** 데이터의 합계와 평균을 구하는 프로그래밍의 기초 패턴을 완성합니다.



Gemini에게 물어보세요

+ ∞

빠른 모델

